

b02 bearbeitet

mborghardt

May 2026

1 Git-Spiel

1. Ändern Sie eine Datei, die im Branch **end** nicht verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem **master** einen neuen Commit. Mergen Sie danach den Branch **end** in den **master**-Branch.

Erst muss ich herausfinden, welche Dateien nicht geändert wurden. Dafür bin ich mit "git checkout end" auf den Branch "end" gewechselt. Mit "git log" habe ich die commit-id für den ersten und den letzten Commit im Branch "end" herausgefunden. So konnte ich mit "git diff <erste-commit-id> <letzte-commit-id>" herausfinden welche Dateien im Branch "end" geändert wurden.

Die Datei shopkeeper.md wurde nicht geändert, also habe ich sie geändert und dann auf dem master Branch mit "git add" und "git commit -m" committed und anschließend mit "git merge end" gemerged.

```
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_1> git ll
* ff4d3ac (HEAD -> master) end in master merged
|
| * 6405bd9 (origin/end) ende
| * 50275b0 tag 20
| * ef5cd63 tag 10
| * 3423367 Änderung in shopkeeper.md
|/
* 76973c1 (origin/master, origin/HEAD) tag 04.5
* 8f88b19 tag 04.4
* b6d614f tag 04.3
```

2. Ändern Sie nun eine Datei, die auch im Branch **end** verändert wurde. Achten Sie dabei darauf, die Änderung an einer anderen Stelle in der Datei vorzunehmen. Erzeugen Sie mit diesen Änderungen auf dem **master** einen neuen Commit. Mergen Sie danach den Branch **end** in den **master**-Branch.

Es passiert eigentlich genau das selbe wie in Aufgabe 1, solange man nicht die selbe Stelle in der Datei ändert, weil dann ein Merge-Konflikt entstehen kann.

```
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_2> git add .\stats.md
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_2> git commit -m "Änderung in stats.md"
[master 4247820] Änderung in stats.md
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_2> git merge end "end in master gemerged"
merge: end in master gemerged - not something we can merge
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_2> git merge end -m "end in master gemerged"
Auto-merging stats.md
Merge made by the 'ort' strategy.
questlog.md | 6 +++++
rucksack.md | 2 +-
stats.md | 6 +----
3 files changed, 10 insertions(+), 4 deletions(-)
PS C:\Users\manue\Documents\2.Semester\Programmieren 2\prog2_ybel_gitquest_b02_2> git ll
* 5011dd4 (HEAD -> master) end in master gemerged
* 6405bd9 (origin/end, end) ende
* 50275b0 tag 20
* ef5cd63 tag 10
* | 4247820 Änderung in stats.md
* 76973c1 (origin/master, origin/HEAD) tag 04.5
* 8f88b19 tag 04.4
* b6d614f tag 04.3
```

3. Wie (2), aber ändern Sie nun eine Stelle, die auch im Branch **end** verändert wurde. Erzeugen Sie mit diesen Änderungen auf dem **master** einen neuen Commit. Mergen Sie danach den Branch **end** in den **master**-Branch. Was passiert, wenn die Änderung im **master** identisch zu der in **end** ist? Was passiert, wenn die Änderung im **master** anders ist als in **end**?

Wenn die Änderungen im master anders sind, als im end Branch, kommt ein Merge-Konflikt und man muss sich entscheiden welche Änderung man beibehalten will, oder ob man eine Kombination aus beiden Änderungen haben will. Wenn die Änderungen in beiden Branches aber identisch ist, wird fehlerfrei gemerged, weil es keine Entscheidung geben kann.

Merge-Konflikt:

```
stats.md ! X
C: > Users > manue > Documents > 2.Semester > Programmieren 2
1  # Stats
2
3  | Property | Value |
4  |-----|-----|
5  Accept Current Change | Accept Incoming Change | Accept Both
6  <<<<<< HEAD (Current Change)
7  | health | 8 |
8  | experience | 40 |
9  =====
10 | health | 2 |
11 | experience | 300 |
12 >>>>>> origin/end (Incoming Change)
13 | hunger | 10 |
14 | weapon | sword (3 dmg) |
15 | armor | light (2 dmg) |
```

4. Wie (2), aber setzen Sie bitte den Branch end auf die Spitze von master, bevor Sie end in master mergen.

Wenn man so vorgeht steht im HEAD "master, end" und nicht nur "master", wie bei den anderen Aufgaben

```
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git add .
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git commit -m "Änderung in master"
[master 3472b7d] Änderung in master
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git checkout end
branch 'end' set up to track 'origin/end'.
Switched to a new branch 'end'
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git merge master
Auto-merging stats.md
Merge made by the 'ort' strategy.
 stats.md | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git checkout master
Switched to branch 'master'
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git merge end
Updating 3472b7d..d8d5d15
Fast-forward
 questlog.md | 6 +++++
 rucksack.md | 2 +-
 stats.md    | 6 +++---
3 files changed, 10 insertions(+), 4 deletions(-)
PS C:\Users\manue\Documents\2. Semester\Prgrammieren 2\prog2_ybel_gitquest_b02_4> git ll
* d8d5d15 (HEAD -> master, end) Merge branch 'master' into end
|
| * 3472b7d Änderung in master
| * 6405bd9 (origin/end) ende
| * 50275b0 tag 20
| * ef5cd63 tag 10
|
| * 76973c1 (origin/master, origin/HEAD) tag 04.5
| * 8f88b19 tag 04.4
| * b6d614f tag 04.3
| * 8ff6931 tag 04.2
```

2 Katzen-Café

2.1 Gradle

```
plugins {  
    id 'java'  
    id("com.diffplug.spotless") version "8.4.0"  
    id 'application'  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation(platform("org.junit:junit-bom:6.0.3"))  
    testImplementation("org.junit.jupiter:junit-jupiter-api")  
    testRuntimeOnly("org.junit.platform:junit-platform-launcher")  
    testRuntimeOnly("org.junit.jupiter:junit-jupiter-engine")  
    testRuntimeOnly("org.junit.vintage:junit-vintage-engine")  
}  
  
application {  
    mainClass = 'catcafe.Main'  
}  
  
test {  
    useJUnitPlatform()  
}  
  
spotless {  
    java { JavaExtension it ->  
        googleJavaFormat()  
    }  
}
```

Mit dieser Konfiguration kann ich die Main-Klasse über Gradle ausführen, JUnit-Tests mit Gradle ausführen und den Code mit google-java-format formatieren. Jedoch gibt es Probleme mit JUnit. Wenn ich die Tests laufen lassen will kommen Fehler die, wie ich glaube nicht an der Konfiguration liegen, sondern an was anderem. Ich hab schon versucht mit Google und LLM's mir helfen zu lassen. Das hat mich aber auch nicht weiter gebracht. Zudem wurde in der Aufgabe gefragt ob man für den Google-Java-Formatter die Einrückung auf 4 Leerzeichen anpassen kann. Das geht aber nur bei eclipse und ich verwende IntelliJ.

Eclipse

The latest version of the `google-java-format` Eclipse plugin can be downloaded from the [releases page](#). Drop it into the Eclipse [drop-ins folder](#) to activate the plugin.

The plugin adds two formatter implementations:

- `google-java-format` : using 2 spaces indent
- `aosp-java-format` : using 4 spaces indent

These that can be selected in "Window" > "Preferences" > "Java" > "Code Style" > "Formatter" > "Formatter Implementation".

2.2 JUnit

Die 10 Testfälle kann man in "CatCafeTest.java" einsehen.

2.2.1 Warum sind die von Ihnen formulierten Testfälle relevant?

Mit diesen Tests werden die Grundfunktionalitäten, wie das hinzufügen und zählen von den Katzen überprüft. Zudem wird überprüft, ob die if-Abfragen funktionieren, wie sie sollen.

2.2.2 Warum halten Sie die formulierten Testfälle für unterschiedlich?

Weil jeder Test entweder eine andere Methode behandelt oder ein unterschiedlichen Bereich in der Methode.

3 Remotes und CI-Pipeline

Den workflow kann man in "ci.yml" einsehen.